# Manual de usuario y manual técnico

**Taller 01: Simulación de MANETs**

**ME01 — Modelos estocásticos y simulación en computación y comunicaciones**

**Universidad Nacional de Colombia**

Este documento tiene dos partes. El manual de usuario está dirigido a quien corre la simulación y lee los resultados sin tocar el código. El manual técnico está dirigido a quien instala, mantiene o modifica la simulación. La simulación está escrita en C++ sobre ns-3 3.46.1 [4] y el análisis en Python; el estimador opcional en Python se conecta con ns3-ai [5].

---

# Parte I. Manual de usuario

## 1. Qué hace el programa

Un perro se pierde con un collar que emite una señal (*beacon*) cada segundo. Los celulares que pasan cerca saben dónde están por GPS y miden con qué fuerza oyen el collar (RSSI). Con esas lecturas, la red ad hoc estima dónde está el perro, y el dueño consulta esa estimación a través de la misma red. Es análogo a "Find My" de Apple, y sirve de excusa para estudiar modelos de movilidad grupal en una MANET jerárquica [1].

La simulación recrea 32 celulares en un área de 500 × 300 m, organizados en una jerarquía con dos niveles de agrupamiento (el primer nivel son los clústeres y el segundo las zonas):

| Nivel | Qué es | Cuántos |
|---|---|---|
| Primer nivel | Clústeres: `bus-norte` y `plaza` en el norte, `bus-sur` y `paseador` en el sur | 4 |
| Segundo nivel | Zonas: `sur` (y de 0 a 150 m) y `norte` (y de 150 a 300 m) | 2 |
| nodo | Celulares, 8 por clúster | 32 |

El perro es un nodo aparte que deambula al azar a 0.3 m/s. La posición de cada celular es la suma de tres capas: la zona, que no se mueve; el clúster, que se mueve como un bloque (los buses siguen una ruta rectangular con paradas, los grupos a pie van de un punto al azar a otro); y el propio celular, que camina al azar dentro de la caja de su clúster.

Dos interruptores encienden o apagan el movimiento de cada nivel, `--clusterMobility` y `--nodeMobility`, y sus cuatro combinaciones son los escenarios de movilidad que se estudian: estático, solo-nodo, solo-clúster y clúster+nodo.

El collar emite por 802.11b. La señal se atenúa con la distancia según un modelo log-distancia [1] y se le suma un sombreado aleatorio, que es la parte estocástica principal y hace que dos lecturas a la misma distancia den RSSI distintos. Cada celular que decodifica el beacon anota el RSSI y su posición GPS (la real más un error aleatorio). Con esas lecturas, cada cabeza de zona estima la posición del perro por uno de tres

métodos (`cercano`, `centroide`, `trilateracion`) y el dueño consulta el resultado por la red, que enruta con AODV [6], OLSR [7] o DSDV [8].

El programa escribe todo lo que pasó (lecturas, posiciones, mensajes, estimaciones y consultas) en archivos CSV dentro de una carpeta de salida. Después, unos scripts de Python leen esa carpeta para estimar fuera de línea, medir la red y dibujar la escena. También hay un script que corre un experimento factorial completo con réplicas.

> Para la pregunta de investigación, las hipótesis y el diseño experimental, ver el literal b (`../b-problema/problema.md`). Para la arquitectura detallada, ver el literal c (`../c-diseno/diseno.md`).

## 2. Qué hay que tener instalado

Para correr las simulaciones y el análisis basta con el entorno que instala `./instalar.sh`:

- ns-3 3.46.1 [4], que el script descarga y compila.
- Un entorno de Python en `.venv/` con `numpy` y `matplotlib`, que el script crea.

Para el estimador en Python (`--estimator=python`) además hacen falta Boost, protobuf y pybind11 al compilar ns-3, porque sostienen el módulo ns3-ai [5]. Sin ellos todo funciona igual, menos esa opción.

Para ver la animación hace falta NetAnim, que se compila aparte y necesita Qt 5.

La instalación paso a paso está en el manual técnico (parte II, sección 2) y en `INSTALACION.md`. Una vez instalado, cada terminal nueva arranca así, desde la raíz del repositorio:

```
source entorno.sh
cd codigo
./simular --help
```

`source entorno.sh` deja la ruta de ns-3 y activa el entorno de Python. Sin él, los comandos avisan "No encuentro ns-3".

## 3. Correr una simulación

Todo se corre desde `codigo/`, con el entorno cargado. Una corrida con los valores por defecto:

```
source entorno.sh
cd codigo
./simular --outDir=resultados/demo
```

`./simular` corre una simulación de 600 s (el valor por defecto de `simTime`) y deja los resultados en `resultados/demo/`. Al terminar imprime un resumen con cuántos beacons, lecturas, mensajes y consultas hubo.

Se pueden cambiar parámetros sueltos en la línea de comandos:

```
./simular --simTime=300 --routing=olsr --shadowSigmaDb=0 --
outDir=resultados/demo2
```

Para un escenario completo, lo cómodo es copiar `escenarios/base.conf`, editar lo que haga falta y pasarlo con `--conf`:

```
cp escenarios/base.conf escenarios/mi-prueba.conf
# editar escenarios/mi-prueba.conf
./simular --conf=escenarios/mi-prueba.conf --outDir=resultados/mi-prueba
```

Un valor escrito en la línea de comandos después de `--conf` le gana al archivo. La lista completa y siempre al día de parámetros sale con `./simular --help`.

Después de correr una simulación, los tres scripts de análisis leen su carpeta:

```
python analisis/localizar.py resultados/demo
python analisis/red.py resultados/demo
python analisis/graficar.py resultados/demo
```

- `localizar.py` estima la posición del perro con todas las lecturas, imprime el error de cada método y escribe `estimaciones.csv` y `resumen.csv`.
- `red.py` imprime cuántos mensajes llegaron y cuánto tardaron, y escribe `resumen_red.csv`.
- `graficar.py` dibuja las trayectorias y las estimaciones en `escena.png` (necesita que antes se haya corrido `localizar.py`, porque lee `estimaciones.csv`).

## 4. Tabla de parámetros

Todos los parámetros están en `codigo/escenarios/base.conf`, con su valor por defecto y un comentario. La tabla siguiente los agrupa igual que el archivo. El rango razonable es orientativo, para explorar escenarios; no es un límite que imponga el programa.

### Movilidad (las variables del factorial)

| Parámetro | Qué controla | Por defecto | Rango razonable |
|---|---|---|---|
| `clusterMobility` | ¿Se mueven los clústeres (nivel 1)? | `true` | `true` / `false` |
| `nodeMobility` | ¿Se mueven los celulares dentro de su clúster? | `true` | `true` / `false` |

### Tamaño y duración

| Parámetro | Qué controla | Por defecto | Rango razonable |
|---|---|---|---|
| `nodesPerCluster` | Celulares por clúster (4 clústeres; mínimo 2: cabeza + dueño) | `8` | 2–20 |
| `simTime` | Duración de la simulación (s) | `600` | 60–1800 |

### Clústeres

| Parámetro | Qué controla | Por defecto | Rango razonable |
|---|---|---|---|
| `busSpeed` | Rapidez de los buses (m/s) | `6` | 3–15 |
| `busDwell` | Parada del bus en cada esquina (s) | `20` | 0–60 |
| `groupSpeedMin` | Rapidez mínima de los grupos a pie (m/s) | `0.5` | 0.1–2 |
| `groupSpeedMax` | Rapidez máxima de los grupos a pie (m/s) | `1.5` | 0.5–3 |
| `groupPause` | Pausa máxima de los grupos entre destinos (s) | `30` | 0–120 |

### Perro perdido

| Parámetro | Qué controla | Por defecto | Rango razonable |
|---|---|---|---|
| `dogMobility` | El perro deambula (si `false`, queda quieto) | `true` | `true` / `false` |
| `dogX` | Posición inicial x del perro (m) | `300` | 0–500 |
| `dogY` | Posición inicial y del perro (m) | `140` | 0–300 |
| `dogSpeed` | Rapidez del perro (m/s) | `0.3` | 0–2 |

### Collar y radio

| Parámetro | Qué controla | Por defecto | Rango razonable |
|---|---|---|---|
| `beaconInterval` | Segundos entre beacons | `1` | 0.5–10 |
| `beaconSize` | Carga útil del beacon (bytes) | `32` | 16–256 |
| `collarTxDbm` | Potencia de transmisión del collar (dBm) | `0` | −10–20 |
| `phoneTxDbm` | Potencia de transmisión de los celulares (dBm) | `16` | 0–20 |
| `rxSensitivityDbm` | Sensibilidad de recepción (dBm) | `-95` | −100 a −80 |

| Parámetro | Qué controla | Por defecto | Rango razonable |
|---|---|---|---|
| `phoneDataMode` | Modo 802.11b de los celulares | `DsssRate1Mbps` | `DsssRate1Mbps` \| `DsssRate2Mbps` \| `DsssRate5_5Mbps` \| `DsssRate11Mbps` |

### Canal (componente estocástico)

| Parámetro | Qué controla | Por defecto | Rango razonable |
|---|---|---|---|
| `pathLossExp` | Exponente de pérdida log-distancia | `2.7` | 2–4 |
| `refLossDb` | Pérdida a 1 m, 2.4 GHz (dB) | `40.05` | — (se deriva de la frecuencia) |
| `shadowSigmaDb` | Desviación del sombreado (dB); `0` = sin sombreado | `4` | 0–8 |
| `gpsSigma` | Desviación del error GPS por eje (m) | `4` | 0–15 |

### Red

| Parámetro | Qué controla | Por defecto | Rango razonable |
|---|---|---|---|
| `routing` | Protocolo de enrutamiento | `aodv` | `aodv` \| `olsr` \| `dsdv` |
| `arpQueue` | Paquetes en espera por vecino sin resolver en ARP (ns-3 trae 3) | `64` | 3–128 |
| `reportJitterMs` | Retardo aleatorio máximo antes de enviar una lectura (ms) | `100` | 0–500 |

### Estimación y consultas

| Parámetro | Qué controla | Por defecto | Rango razonable |
|---|---|---|---|
| `estMethod` | Método de estimación en la cabeza de zona | `trilateracion` | `cercano` \| `centroide` \| `trilateracion` |
| `estimator` | Dónde corre el estimador (C++ o Python por ns3-ai) | `cpp` | `cpp` \| `python` |
| `estWindow` | Segundos de lecturas que usa cada estimación | `10` | 5–30 |
| `estInterval` | Segundos entre estimaciones de cada cabeza de zona | `5` | 1–30 |
| `zoneFusion` | Cómo combinar las dos zonas | `none` | `none` \| `raw` \| `merge` |

| Parámetro | Qué controla | Por defecto | Rango razonable |
|---|---|---|---|
| `queryInterval` | Segundos promedio entre consultas del dueño (uniforme entre 0.5× y 1.5×) | `10` | 1–60 |
| `queryStart` | Primera consulta del dueño (s) | `20` | 0–`simTime` |
| `ownerCluster` | Clúster del dueño | `0` | `0` bus-norte \| `1` plaza \| `2` bus-sur \| `3` paseador |

Salida

| Parámetro | Qué controla | Por defecto | Rango razonable |
|---|---|---|---|
| `posLogInterval` | Segundos entre registros de posiciones | `1` | 0.5–10 |
| `netanim` | Escribir `netanim.xml` para la animación | `false` | `true` / `false` |

> Nota: `estimator=python` solo surte efecto cuando la corrida se lanza con `analisis/ns3ai_estimador.py` (sección 5), que es quien crea la memoria compartida; pasarlo a `./simular` directamente aborta la corrida pidiendo ese lanzador.

## 5. Correr el experimento completo

`analisis/experimento.py` corre el diseño factorial: las cuatro combinaciones de movilidad, con cada protocolo de enrutamiento, repetidas varias veces con semillas distintas, y en paralelo. Desde `codigo/`:

```
python analisis/experimento.py --nombre prueba --reps 10 --routing aodv
olsr dsdv -- --simTime=600
```

Esto corre las 4 combinaciones × 3 protocolos × 10 réplicas = 120 corridas y deja las tablas en `resultados/prueba/`. Lo que va después de `--` se pasa tal cual a `collar-sim`, con los mismos valores para todas las combinaciones. Opciones del script:

- `--nombre`: subcarpeta de `resultados/`.
- `--reps`: réplicas por combinación (por defecto 10).
- `--routing`: uno o más de `aodv`, `olsr`, `dsdv` (por defecto solo `aodv`).
- `--paso` y `--ventana`: cada cuántos segundos estima el análisis fuera de línea y con qué ventana (por defecto 5 y 10 s).
- `--estimador`: `cpp` (por defecto) o `python`.
- `--jobs`: corridas en paralelo (por defecto, núcleos − 1; con `python`, la mitad, porque cada corrida ocupa dos núcleos mientras espera).
- `--sin-compilar`: no recompilar antes de correr.

El script recompila el binario solo, corre todo, y para cada corrida calcula la localización de referencia (con `localizar.py`) y las métricas de red (con `red.py`). Al final escribe dos tablas, `experimento.csv` (una fila por corrida y método) y `experimento_red.csv` (una fila por corrida), guarda el comando en

`comando.txt` e imprime tablas resumen de error de localización, entrega, latencia y calidad de las consultas, con media ± desviación entre réplicas.

Para usar un escenario propio en el experimento, se pasa el `.conf` después de `--`:

```
python analisis/experimento.py --nombre mi-prueba --reps 10 -- --
conf=escenarios/mi-prueba.conf
```

Los factores del diseño (movilidad de clúster, movilidad de nodo, protocolo y semilla) los pone el script y siempre le ganan a lo que diga el `.conf`.

### Experimento con el estimador en Python

```
python analisis/ns3ai_estimador.py -- --outDir=resultados/demo-py
python analisis/experimento.py --nombre prueba-py --reps 10 --estimador
python
```

`ns3ai_estimador.py` lanza una sola corrida con el estimador en Python; `experimento.py --estimador python` corre el factorial en ese modo.

## 6. Qué trae cada archivo de salida

Cada corrida deja en su carpeta `--outDir`:

| Archivo | Qué trae |
|---|---|
| `rssi.csv` | Cada lectura del beacon: quién la tomó, su posición real y GPS, dónde estaba el perro, el RSSI y la distancia real |
| `posiciones.csv` | La posición de cada nodo cada `posLogInterval` segundos (1 s por defecto) |
| `nodos.csv` | Clúster, zona y papel de cada nodo (cabeza de clúster, cabeza de zona, dueño) |
| `mensajes.csv` | Cada mensaje de aplicación (R, V, E, Q, A, X) con la hora de envío y la de llegada; la de llegada queda vacía si se perdió |
| `reportes.csv` | Cada lectura con la hora en que llegó a la cabeza de su zona |
| `estimaciones_red.csv` | Cada estimación de las cabezas de zona, con su error contra la posición real |
| `consultas.csv` | Cada consulta del dueño: si tuvo respuesta, cuánto tardó, qué tan vieja y qué tan errada estaba la estimación |
| `parametros.txt` | Parámetros numéricos y la semilla de la corrida |
| `parametros_texto.txt` | Parámetros de texto (routing, método, etc.), el o los `.conf` usados |

| Archivo | Qué trae |
|---|---|
| `netanim.xml` | La animación para NetAnim, solo con `--netanim=true` |

Los scripts de análisis agregan a esa misma carpeta: `estimaciones.csv` y `resumen.csv` (de `localizar.py`), `resumen_red.csv` (de `red.py`) y `escena.png` (de `graficar.py`).

## 7. Ver la animación en NetAnim

Hay que correr la simulación con `--netanim=true` para que escriba `netanim.xml`:

```
./simular --netanim=true --outDir=resultados/demo
```

Luego se abre NetAnim (ver la instalación en la parte II, sección 2.4) y, dentro del programa, se abre el `netanim.xml` de la carpeta de resultados. En la animación, el collar aparece en rojo y los nodos con el color de su clúster; las cabezas de zona se dibujan más grandes y las etiquetas marcan cabeza de clúster, cabeza de zona y dueño.

## 8. Problemas frecuentes

| Mensaje o síntoma | Qué hacer |
|---|---|
| `No encuentro ns-3` | Falta `source entorno.sh` en esa terminal, o no se ha corrido `./instalar.sh`. |
| `La simulación no está compilada` | `./compilar` desde `codigo/`. |
| `--estimator=python necesita ns3-ai` | ns-3 se compiló sin ns3-ai (faltaba Boost, protobuf o pybind11). Instálenlos y corran `./instalar.sh` otra vez. |
| Una corrida con el estimador en Python que no termina | El lanzador corta la espera si la simulación muere sin avisar. Si aun así queda algo corriendo, `pkill -f collar-sim` lo detiene. |
| `graficar.py` falla porque falta `estimaciones.csv` | Corran antes `python analisis/localizar.py <carpeta>`. |
| La simulación aborta con un mensaje de parámetro inválido | Revisen el valor: `routing` debe ser `aodv`/`olsr`/`dsdv`, `estMethod` uno de los tres métodos, `zoneFusion` `none`/`raw`/`merge`, `nodesPerCluster` ≥ 2. |

Los problemas de instalación (macOS, WSL, NetAnim) están en la parte II y en `INSTALACION.md`.

---

# Parte II. Manual técnico

## 1. Arquitectura

La simulación en C++ (`codigo/collar-sim.cc`) corre sobre ns-3 3.46.1 [4] y, al terminar, escribe varios CSV en la carpeta `--outDir`. Los scripts de Python en `codigo/analisis/` leen esos CSV para estimar fuera de línea, medir la red y graficar. Con `--estimator=python`, además, Python participa durante la simulación: ns-3 le pasa cada ventana de lecturas por memoria compartida (ns3-ai [5]) y sigue con la posición que Python devuelve.

El diseño detallado, con los diagramas de arquitectura, jerarquía, movilidad, secuencia de mensajes y los flujos de la cabeza de zona, del estimador y del experimento, está en el literal c (`../c-diseno/diseno.md`). Esta sección y las siguientes describen el código que realiza ese diseño.

## 2. Instalación de ns-3, ns3-ai y NetAnim

Los pasos completos y los errores conocidos están en `INSTALACION.md`. Sirve para macOS y para Ubuntu (en Windows, Ubuntu dentro de WSL, porque ns-3 no compila en Windows directamente). Resumen:

### 2.1 Dependencias del sistema

macOS, con Homebrew:

```
xcode-select --install
brew install cmake ninja python boost protobuf pybind11
```

Ubuntu, o Ubuntu en WSL:

```
sudo apt update
sudo apt install g++ cmake ninja-build python3 python3-dev python3-venv
curl bzip2 git \
    libboost-all-dev libprotobuf-dev protobuf-compiler pybind11-dev
```

Boost, protobuf y pybind11 son para ns3-ai [5]; sin ellos todo funciona menos `--estimator=python`.

### 2.2 El instalador

```
git clone https://github.com/mbuitragocUNAL/ME01-taller01-manets.git
cd ME01-taller01-manets
./instalar.sh
```

El script, en orden: revisa las dependencias; descarga ns-allinone-3.46.1 (~80 MB) y lo descomprime; crea el entorno de Python en `.venv/` con numpy y matplotlib; enlaza `codigo/` dentro de ns-3 en `scratch/manets`; configura ns-3 con los módulos que se usan (y ns3-ai si están sus dependencias) y compila; escribe `entorno.sh` con la ruta de ns-3 y la activación del entorno; corre una simulación de 30 s y, si hay ns3-ai, otra con el estimador en Python. ns-3 queda en `~/ns3` por defecto; para otra ruta, `./instalar.sh ~/otra/carpeta`. Se puede volver a correr sin repetir lo ya hecho.

### 2.3 Uso diario

En cada terminal nueva, desde la raíz del repositorio: `source entorno.sh`, luego `cd codigo`. Después de cambiar un `.cc` o un `.h`, `./compilar`.

### 2.4 NetAnim (opcional)

NetAnim no viene en el paquete de ns-3 desde la 3.45, así que se compila aparte; necesita Qt 5.

```
# macOS
brew install qt@5
# Ubuntu
sudo apt install qtbase5-dev qt5-qmake
```

```
git clone https://gitlab.com/nsnam/netanim.git ~/netanim
cd ~/netanim
git checkout netanim-3.110
mkdir build && cd build
cmake .. -G Ninja -DCMAKE_PREFIX_PATH="$(brew --prefix qt@5 2>/dev/null)" -
DCMAKE_DISABLE_FIND_PACKAGE_Qt6=ON
ninja
./bin/netanim
```

En Ubuntu, `brew --prefix` no existe y esa parte queda vacía, que es justo lo que se necesita.

> Notas de verificación: según `INSTALACION.md`, la instalación completa se probó el 29 de octubre de 2026 en un Mac con Apple silicon (unos 2 minutos con 13 núcleos; 5 a 10 minutos en 4 núcleos). En Ubuntu/WSL aún no la había probado nadie al escribir esto, así que el grupo debería confirmar esos pasos antes de la entrega. Los errores conocidos de macOS (enlazador `lld`, archivo `messages.pb.h` en una carrera de compilación) están en la tabla de `INSTALACION.md`.

## 3. Cómo se compila y qué hace `CMakeLists.txt`

`./compilar` carga `entorno.sh` si hace falta y corre, dentro de la carpeta de ns-3:

```
./ns3 build scratch/manets/collar-sim
```

`codigo/` está enlazado en ns-3 como `scratch/manets`. El archivo `codigo/CMakeLists.txt` existe para que ns-3 **no** arme el programa solo, y poder compilar además el binding de Python de ns3-ai:

- Define la variable `COLLAR_CON_NS3AI` solo si ns-3 compiló el módulo `ai` (es decir, si estaban Boost, protobuf y pybind11). El `#ifdef COLLAR_CON_NS3AI` del código activa o desactiva toda la parte de ns3-ai según eso; sin el módulo, la simulación compila igual, solo que sin `--estimator=python`.
- Arma el ejecutable `collar-sim` con `build_exec(...)`, a partir de `collar-sim.cc` y los encabezados `estimador.h` y `ns3ai/collar-msg.h`.
- Si hay ns3-ai, compila el binding de Python `collar_py.cc` con `pybind11_add_module` en el módulo `collar_ns3ai_py`, lo deja en `codigo/ns3ai/` y lo marca como dependencia del

ejecutable. En macOS lo enlaza contra `core` de ns-3, porque sin eso el `import` falla con "symbol not found … ns3::Time::StaticInit".

El binario final queda en `build/scratch/manets/`, con un nombre que depende de la versión y del perfil de compilación (por ejemplo `ns3.46.1-collar-sim-default`). Por eso los lanzadores `./simular` y `rutas.py` buscan el más reciente con un patrón, en vez de un nombre fijo.

## 4. Estructura del código, archivo por archivo

```
codigo/
├── CMakeLists.txt           arma collar-sim y, si hay ns3-ai, el binding
de Python
├── compilar                 ./ns3 build scratch/manets/collar-sim
├── simular                  corre el binario compilado con los argumentos
dados
├── collar-sim.cc            la simulación completa
├── estimador.h              los tres métodos de estimación en C++
├── escenarios/
│   └── base.conf            todos los parámetros con su valor por defecto
├── ns3ai/
│   ├── collar-msg.h         structs Peticion/Respuesta de la memoria
compartida
│   └── collar_py.cc         binding pybind11 de esos structs y de la
interfaz ns3-ai
└── analisis/
    ├── localizar.py         estimación de referencia fuera de línea; ->
estimaciones.csv, resumen.csv
    ├── red.py               métricas de red (entrega, latencia,
consultas); -> resumen_red.csv
    ├── graficar.py          dibuja trayectorias y estimaciones; ->
escena.png
    ├── experimento.py       corre el factorial con réplicas, en paralelo;
-> experimento*.csv
    ├── ns3ai_estimador.py   lanza una corrida con el estimador en Python
    └── rutas.py             rutas compartidas (NS3_DIR, binario) para los
scripts
```

- `collar-sim.cc`: arma los nodos, la movilidad jerárquica, el Wi-Fi, la pila IP con el enrutamiento, las aplicaciones y los *traces*; corre la simulación y escribe los CSV. Las secciones 5 y 6 lo detallan.
- `estimador.h`: biblioteca de cabecera con `collar::Estimar(...)` y los métodos `Centroide` y `Trilaterar`. La usan la simulación (cabezas de zona en C++) y, por espejo, `localizar.py`.
- `escenarios/base.conf`: fuente de la tabla de parámetros de la parte I.
- `ns3ai/`: todo lo de la integración con Python (sección 6).
- `analisis/`: los scripts de post-proceso y el experimento.

## 5. El modelo

### 5.1 Jerarquía

La posición absoluta de cada celular es la suma de tres capas, armada con `HierarchicalMobilityModel`:

| Nivel | Qué es | Cuántos | Modelo de movilidad |
|---|---|---|---|
| Primer nivel | Clústeres | 4 | bus → `WaypointMobilityModel`; grupo → `RandomWaypointMobilityModel` |
| Segundo nivel | Zonas: `sur` (y ∈ [0,150)), `norte` (y ∈ [150,300)) | 2 | `ConstantPositionMobilityModel` (origen de la zona) |
| nodo | Celulares, 8 por clúster | 32 | `RandomWalk2dMobilityModel` dentro de la caja del clúster |

El clúster se arma como un `HierarchicalMobilityModel` con `SetParent(zona)` y `SetChild(movilidad local)`; cada nodo agrega otro `HierarchicalMobilityModel` con el clúster como padre. Los clústeres están definidos en `kClusters`:

| Clúster | Zona | Tipo | Caja de nodos (semi-ejes) | Rapidez de nodo |
|---|---|---|---|---|
| `bus-norte` | norte | BUS | 6.0 × 1.5 m | 0.5 m/s |
| `plaza` | norte | GROUP | 10.0 × 10.0 m | 1.0 m/s |
| `bus-sur` | sur | BUS | 6.0 × 1.5 m | 0.5 m/s |
| `paseador` | sur | GROUP | 3.0 × 3.0 m | 0.8 m/s |

El área total es 500 × 300 m. Las cajas que menciona el README (12 × 3, 20 × 20, 6 × 6 m) son los anchos completos, es decir el doble de los semi-ejes. La cabeza de cada clúster es su nodo 0. La cabeza de zona norte es la de `plaza` y la de zona sur la de `paseador` (`kZones` define `headCluster`). El dueño es el nodo 1 del clúster `ownerCluster` (por defecto `bus-norte`). El perro es un nodo aparte, no pertenece a ningún clúster, con `RandomWalk2dMobilityModel` a `dogSpeed` (o fijo si `dogMobility=false`).

## 5.2 Movilidad

- Bus (`MakeBusRoute`): `WaypointMobilityModel` que recorre en bucle la ruta rectangular `kBusRoute`, (50,30)→(450,30)→(450,120)→(50,120) relativa al origen de su zona, a `busSpeed`, con una parada de `busDwell` en cada esquina.
- Grupo: `RandomWaypointMobilityModel` que va de un punto al azar a otro dentro de la zona, con rapidez uniforme entre `groupSpeedMin` y `groupSpeedMax` y pausa uniforme hasta `groupPause`.
- Nodo: `RandomWalk2dMobilityModel` acotado a la caja del clúster, a rapidez constante `nodeSpeed`, que cambia de dirección cada 5 s.
- Con `clusterMobility=false` el clúster queda quieto en su posición inicial (el bus en su esquina de arranque); con `nodeMobility=false` cada nodo queda quieto en su posición dentro de la caja.

## 5.3 Canal

El collar emite por 802.11b ad hoc (`AdhocWifiMac`, `WIFI_STANDARD_80211b`), un beacon broadcast cada `beaconInterval` (con un pequeño *jitter*) de `beaconSize` bytes a `collarTxDbm`. El canal

(YansWifiChannel) usa:

- LogDistancePropagationLossModel con exponente pathLossExp y pérdida de referencia refLossDb a 1 m [1].
- Si shadowSigmaDb > 0, un RandomPropagationLossModel con variable normal de media 0 y varianza shadowSigmaDb² en dB: el sombreado log-normal, la aleatoriedad clave del modelo. Con shadowSigmaDb=0 no se agrega.

El modelo de RSSI es:

$$P_r(d) = P_0 - 10\,n\,\log_{10}(d) + X, \qquad X \sim \mathcal{N}(0, \sigma^2)$$

con $P_0 = \mathbf{collarTxDbm} - \mathbf{refLossDb}$, $n = \mathbf{pathLossExp}$ y $\sigma = \mathbf{shadowSigmaDb}$. Cada celular que decodifica el beacon (trace MonitorSnifferRx, en SnifferRx) registra el RSSI (signalNoise.signal) y su posición GPS, que es la real más un ruido normal de desviación gpsSigma por eje.

## 5.4 Mensajes

La aplicación corre sobre UDP por encima del enrutamiento elegido. Los tipos (de HandleMsg):

| Tipo | Nombre | De → a | Cuándo |
|---|---|---|---|
| R | reporte | celular → cabeza de su clúster | cada lectura, tras un jitter de 0 a reportJitterMs |
| V | relevo | cabeza de clúster → cabeza de zona | al recibir un reporte |
| E | intercambio | cabeza de zona → otra cabeza de zona | cada estimación (estInterval) |
| Q | consulta | dueño → cabeza de su zona | cada queryInterval (uniforme 0.5×–1.5×) |
| A | respuesta | cabeza de zona → dueño | al recibir una consulta |
| X | cruda | cabeza de zona → otra cabeza de zona | cada lectura, solo con zoneFusion=raw |

Un R que llega a un nodo que no es cabeza de zona se reenvía como V hacia la cabeza de zona; si el nodo ya es cabeza de zona, se entrega localmente (DeliverReport). SendMsg entrega localmente cuando origen = destino. Las cabezas de zona acumulan las lecturas de la ventana en lecturas/lecturaT.

## 5.5 Estimación

Cada estInterval (EstimateTick), cada cabeza de zona descarta las lecturas más viejas que estWindow y estima con el resto. Los métodos (estMethod, en estimador.h):

- cercano: posición GPS del celular con mayor RSSI (línea base tipo Find My).
- centroide: centroide de posiciones GPS ponderado por potencia lineal (mW), con peso $w = 10^{\mathrm{RSSI}/10}$.
- trilateracion: convierte cada RSSI en distancia $\hat{d} = 10^{(P_0 - \mathrm{RSSI})/(10n)}$ y resuelve mínimos cuadrados no lineales con pesos $1/\hat{d}^2$ por Gauss-Newton, partiendo del centroide. Requiere al menos

3 celulares distintos. Si la geometría es degenerada (número de condición de la matriz normal 2×2 por encima de 1e8), no devuelve estimación.

La fusión de zonas (`zoneFusion`, en `BestEstimate`):

- `none` (por defecto): cada cabeza estima solo con su zona; al dueño le llega la estimación con más observadores, o la más reciente si una ya supera `estWindow`.
- `raw`: cada cabeza reenvía las lecturas crudas de su zona a la otra (mensaje X); ambas estiman con todo.
- `merge`: cada una estima por su lado y, al contestar, se promedian ponderando por número de observadores.

El mismo método está en C++ (`estimador.h`) y en Python (`localizar.py`), para que la estimación dentro de la red y la de referencia fuera de línea sean comparables.

## 6. Integración con ns3-ai

Con `--estimator=python`, en cada estimación la simulación se detiene, pasa la ventana de lecturas a Python por memoria compartida y sigue con la posición que Python devuelve. Las piezas:

- `ns3ai/collar-msg.h`: define los structs `collar::Peticion` (método, zona, cabeza de zona, instante, $P_0$, $n$, número de lecturas y arreglos de observador, tiempo, x, y, RSSI) y `collar::Respuesta` (válida, x, y). El layout debe ser idéntico en los dos lados. El tope de lecturas por ventana es `kMaxLecturas = 512`; si hay más, van las más recientes (y la corrida cuenta cuántas ventanas truncó, en `ventanasTruncadas`).
- `ns3ai/collar_py.cc`: binding pybind11 que expone `Peticion`, `Respuesta` y la interfaz `Ns3AiMsgInterfaceImpl` al módulo de Python `collar_ns3ai_py`. Los arreglos se exponen como vistas numpy de los primeros `num` elementos. Las esperas de recepción/envío sueltan el GIL para que un hilo vigilante pueda detectar que la simulación murió.
- Lado C++ (`EstimarEnPython` en `collar-sim.cc`, bajo `#ifdef COLLAR_CON_NS3AI`): copia la ventana a la `Peticion`, `CppSendBegin/End` para entregarla, `CppRecvBegin/End` para leer la `Respuesta`. Cuando `estimator=python`, `EstimateTick` llama a `EstimarEnPython` en vez de `collar::Estimar`.
- Ciclo de Python (`ns3ai_estimador.py`): crea la memoria compartida (un bloque de 1 MiB, porque la `Peticion` ocupa ~19 KB y el valor por defecto de ns3-ai de 4 KB no alcanza), lanza `collar-sim` con `--estimator=python` y un `--ns3aiTrial` único, y entra en el bucle: `PyRecvBegin`; si `PyGetFinished`, sale; si no, estima con `estimar_peticion()`, que reconstruye la ventana en el formato de `localizar.py` y llama a la misma `estimar()`, y devuelve la posición con `PySendBegin/End`. Un hilo vigilante corta la espera si el proceso de la simulación muere sin cerrar ns3-ai.

Para cambiar el estimador (por ejemplo a un filtro de Kalman o a un modelo entrenado) basta con editar `estimar_peticion()` en `ns3ai_estimador.py`, sin tocar el C++. Según el README, una corrida de 300 s con el estimador en C++ y con el de Python dio archivos idénticos salvo diferencias de a lo sumo 3 cm en las estimaciones. Cada corrida con Python ocupa dos núcleos al 100 % mientras espera, por eso `experimento.py` corre la mitad de corridas a la vez en ese modo.

El nombre de la memoria compartida (`--ns3aiTrial`) es único por corrida, así que varias corridas con Python pueden ir en paralelo. Al salir, la simulación no vuelve a llamar `CppSetFinished()`: lo hace el

destructor de la interfaz ns3-ai, y una segunda llamada se quedaría esperando a Python para siempre.

# 7. Cómo agregar un estimador, un clúster o un protocolo

## Un estimador

- Sin recompilar C++, solo para el modo Python: cambiar `estimar_peticion()` en `analisis/ns3ai_estimador.py`. Recibe la `Peticion` (con `p.x()`, `p.y()`, `p.rssi()`, `p.p0`, `p.n`, etc.) y devuelve una posición `(x, y)` o `None`. Correr con `ns3ai_estimador.py` o `experimento.py --estimador python`.
- En C++, para que lo usen las cabezas de zona por defecto: agregar un valor al enum `collar::Metodo` y su rama en `collar::Estimar` (`estimador.h`), reconocerlo en `MetodoDesdeTexto`, y si se quiere que el análisis fuera de línea lo tenga, replicarlo en `localizar.py` (función `estimar`) y en la tupla `METODOS`. Recompilar con `./compilar`.

## Un clúster o una zona

Editar las tablas `kZones` y `kClusters` en `collar-sim.cc`. Un clúster nuevo necesita su zona, tipo (`BUS`/`GROUP`), posición de arranque, semi-ejes de la caja, rapidez de nodo y color de NetAnim. Hay que cuidar que `headCluster` de cada zona siga apuntando a un clúster existente y que `ownerCluster` siga en rango; varias partes recorren `kClusters` por índice. Los lanzadores y el análisis no asumen un número fijo de clústeres, pero `graficar.py` y `red.py` traen nombres de zona fijos (`sur`/`norte`) que habría que ampliar si se agregan zonas. Recompilar con `./compilar`.

## Un protocolo de enrutamiento

En `collar-sim.cc`, en el bloque de IP y enrutamiento, agregar una rama al `if (routing == ...)` con el helper del protocolo (como las de `aodv`, `olsr`, `dsdv`), instalarlo en `observers` y, si aplica, asignarle flujos aleatorios. Añadir el nombre a la validación. Recompilar con `./compilar`.

# 8. Reproducibilidad: semillas y archivos de parámetros

- La simulación toma flujos aleatorios de un índice `stream` que arranca en 1000; cada modelo de movilidad, el canal, el enrutamiento y los generadores de jitter/GPS reservan sus propios flujos con `AssignStreams`/`SetStream`, para que no se pisen entre corridas ni entre réplicas.
- La réplica se fija con `--RngRun` (en `experimento.py`, `--RngRun = 1..reps`). Cada corrida guarda `RngRun` y `RngSeed` en `parametros.txt`.
- Cada corrida escribe en su carpeta `parametros.txt` (numéricos y semilla) y `parametros_texto.txt` (texto y el o los `.conf` usados), así que cualquier resultado se puede repetir corriendo con los mismos valores y la misma semilla. `localizar.py` y `red.py` leen esos archivos para recuperar $P_0$, el exponente y `simTime`.
- `experimento.py` guarda además `comando.txt` con la línea exacta que lo lanzó.
- La precedencia de parámetros es: `base.conf` < lo que venga después en la línea de comandos; y en `experimento.py`/`ns3ai_estimador.py` los factores del diseño (y `estimator`/`ns3aiTrial`) van al final y ganan sobre cualquier `.conf`.

# 9. Problemas conocidos y decisiones

Estos efectos se observaron al probar el código durante el desarrollo. Son supuestos o artefactos del modelo que el análisis debe tener en cuenta, no conclusiones; los porcentajes son de pruebas de desarrollo, no resultados formales (esos son del literal f).

- Sesgo del RSSI lejano. Lejos del collar solo se decodifican los beacons con sombreado favorable, así que las lecturas lejanas salen más fuertes de lo que deberían: ajustar RSSI contra distancia da un exponente ~1.6 en vez de 2.7, y la trilateración subestima distancias. Con `shadowSigmaDb=0` el ajuste da 2.7 exacto.
- Sombreado sin correlación. El sombreado se sortea independiente por paquete; en la realidad está correlacionado en espacio y tiempo, así que promediar lecturas ayuda más en la simulación que en la vida real.
- Cola ARP. ns-3 guarda solo 3 paquetes por vecino mientras resuelve ARP; los relevos salen en ráfaga y con AODV se perdía ~40 %. Subir la cola a 64 (`arpQueue`, cerca de los ~100 de Linux) subió la entrega con AODV de 57 % a 93 %. Por eso el valor por defecto de `arpQueue` es 64.
- Sesgo de `zoneFusion=none`. Con `none`, cada cabeza solo ve su zona y su estimación se sesga hacia ella. Se nota en `escena.png`.
- Consultas sin respuesta con AODV. Con AODV, ~22 % de las consultas del dueño quedaron sin respuesta: las rutas de AODV vencen a los 3 s y las consultas llegan cada ~10 s, así que casi cada consulta tiene que rehacer la ruta. Con OLSR y DSDV fue del 1 % al 7 %.
- Costo de `zoneFusion=raw`. `raw` duplica las lecturas de cada cabeza, pero el tráfico extra bajó la entrega de relevos de 99 % a 67 % en una prueba con DSDV.
- 802.11b en vez de Bluetooth. Un collar real usaría Bluetooth, pero ns-3 no lo trae; se usa 802.11b como sustituto.

Decisiones de diseño que vale la pena recordar: las dos alternativas a `zoneFusion=none` (`raw` y `merge`) se dejaron ambas porque parecían igual de válidas; el intervalo de consulta es uniforme entre 0.5× y 1.5× `queryInterval` para que la edad de la respuesta no salga constante; y el estimador se mantiene espejado entre C++ y Python para poder comparar las dos vías.

---

## Referencias


- [1] S. K. Sarkar, T. G. Basavaraju, y C. Puttamadappa, *Ad Hoc Mobile Wireless Networks: Principles, Protocols, and Applications*, 2.ª ed. Boca Raton, FL, EE. UU.: CRC Press, 2013.
- [4] nsnam, "ns-3 Network Simulator", versión 3.46. [En línea]. Disponible: https://www.nsnam.org/
- [5] H. Yin *et al.*, "ns3-ai: Fostering Artificial Intelligence Algorithms for Networking Research", en *Proc. Workshop on ns-3 (WNS3)*, 2020. [En línea]. Disponible: https://github.com/hust-diangroup/ns3-ai
- [6] C. Perkins, E. Belding-Royer, y S. Das, "Ad hoc On-Demand Distance Vector (AODV) Routing", IETF, RFC 3561, jul. 2003.
- [7] T. Clausen y P. Jacquet, "Optimized Link State Routing Protocol (OLSR)", IETF, RFC 3626, oct. 2003.
- [8] C. E. Perkins y P. Bhagwat, "Highly dynamic Destination-Sequenced Distance-Vector routing (DSDV) for mobile computers", en *Proc. ACM SIGCOMM*, 1994, pp. 234–244.